

İçindekiler

proje-kiti — Ne Yapıyor, Nasıl Yapıyor

Bölüm 1 — Bu nedir

Bölüm 2 — Kitin ana fikri

Bölüm 3 — Kullandığı teknolojiler ve neden

Bölüm 4 — /yeni-proje sekiz adımda ne yapar

Bölüm 5 — On dokuz kural dosyası

Bölüm 6 — Yapay zekânın uyduğu kapılar

Bölüm 7 — Kit nasıl büyüyor

Sözlük

proje-kiti — Ne Yapıyor, Nasıl Yapıyor

Sürüm: 1.76.0 · **Tarih:** 2026-09-04 **Depo:** github.com/bariskose9/bariskose-skills

Terimler ilk geçtikleri yerde açıklanır. Sonda toplu bir sözlük vardır.

Bölüm 1 — Bu nedir

Claude Code, terminalden çalışan bir yapay zekâ geliştirme aracıdır. Dosyaları okur, kod yazar, komut çalıştırır. Terminal, bilgisayara yazıyla komut verdiğin siyah ekran.

Skill (yetenek), Claude'a "şu iş şöyle yapılır" diyen yazılı bir talimat dosyasıdır. **Plugin** (eklenti), birkaç skill'in bir arada paketlenmiş hâli. **Marketplace** (mağaza), eklentilerin durduğu ve kurulduğu yer.

proje-kiti bir plugin. İçinde dört skill var:

Komut	Ne yapar
<code>/yeni-proje</code>	Boş klasörden başlayıp internette yayında bir proje çıkarır
<code>/kit-senkron</code>	Projede öğrenilen bir kuralı kalıcı olarak kite yazar
<code>/video-analiz</code>	YouTube videosundan kite eksik olan bilgiyi çıkarır
<code>/pdf-uret</code>	Yazılı belgeyi telefonda okunacak PDF'e çevirir

Ne vaat ediyor, ne vaat etmiyor

Vaat ediyor: doğru kurulmuş, test edilmiş, canlıda çalışan bir proje ve net bir yol haritası.

Vaat etmiyor: "tek cümleyle bitmiş uygulama". Kurulum biter, sonra özellikler adım adım yazılır ve **her adımda plan sunulup onay beklenir**.

Bölüm 2 — Kitin ana fikri

Yazılım projelerinde hatalar genelde kod yazarken değil, **karar verirken** yapılır. Yanlış veritabanı seçilir, yanlış klasör yapısı kurulur, güvenlik sonraya bırakılır. Bunlar sonradan düzeltilemez — düzeltmek projeyi baştan yazmak demektir.

Kit bu kararların hepsini **önceden yazılı kurala** bağlar. Kurallar `docs/standards/` klasöründe 19 dosyada duruyor ve her yeni projeye olduğu gibi kopyalanıyor.

İkinci ana fikir: yapay zekânın hafızası yoktur. Her yeni sohbet sıfırdan başlar. Bu yüzden bilinmesi gereken her şey **dosyaya yazılır** — sohbete değil. Sonraki oturum dosyaları okuyup kaldığı yerden devam eder.

Bölüm 3 — Kullandığı teknolojiler ve neden

Kit **varsayılan bir teknoloji seti** ile gelir. Bu bir başlangıç noktasıdır; her projede yeniden ölçülür ve gerekirse değiştirilir.

Arayüz tarafı — kullanıcının gördüğü kısım

Ne	Nedir	Neden
React	Ekranı parçalara (bileşen) bölerek yazmayı sağlayan kütüphane	Fiili standart; bulması kolay, bilen çok
Next.js	React'in üstüne sayfa yönlendirme, sunucu tarafı çalışma ve optimizasyon ekleyen çatı (framework)	Arama motorunun görebileceği sayfa üretir; React tek başına üretemez
TypeScript	JavaScript'in tip denetimli hâli. " <i>Bu değişken metin, şu sayı</i> " der ve uymayan kodu derlenmeden yakalar	Hatanın kullanıcıya değil sana çıkması için
Tailwind CSS	Görünümü, HTML'in içine yazılan kısa sınıf adlarıyla ayarlayan araç	Ayrı stil dosyası yönetmeden tutarlı görünüm
shadcn/ui	Hazır bileşenler (buton, form, tablo) — paket olarak değil, kodu projeye kopyalanarak gelir	Kopyalandığı için istediğin gibi değiştirirsin; paket güncellemesi tasarımını bozmaz

Framework (çatı): hazır iskelet. Sıfırdan kurmak yerine hazır yapının içine kendi kodunu yazarsın.

Sunucu tarafı — kullanıcının görmediği kısım

Ne	Nedir	Neden
PostgreSQL	İlişkisel veritabanı. Veriler tablolarda durur, tablolar birbirine bağlanır	Başvuru, kayıt, randevu gibi işler doğası gereği ilişkisel
Prisma	ORM — kod ile veritabanı arasındaki çevirmen. SQL yazmak yerine <code>prisma.kullanici.findMany()</code> yazarsın	Şema tek dosyada okunur; yazım hatasını derleme anında yakalar
Zod	Gelen verinin beklenen biçimde olduğunu kontrol eden kütüphane	Dışarıdan gelen hiçbir veriye güvenilmez
NestJS	Ayrı bir sunucu uygulaması yazmak gerektiğinde kullanılan çatı	Yalnızca gerekiyorsa; çoğu projede Next.js tek başına yeter

ORM (Object-Relational Mapping): veritabanı tablolarını koddaki nesnelere eşleyen katman.

İlişkisel veritabanı: verinin tablolara bölündüğü ve tabloların birbirine bağlandığı yapı. Örnek: `kullanıcılar` tablosu ve `randevular` tablosu; her randevu bir kullanıcıya bağlıdır.

Yayın tarafı

Ne	Nedir
Vercel	Next.js projelerini internette yayınlayan servis
Docker	Uygulamayı, çalışması için gereken her şeyle birlikte bir kutuya (container/konteyner) koyan araç. "Bende çalışıyordu" sorununu bitirir
GitHub Actions	Her kod değişikliğinde testleri otomatik çalıştıran sistem (CI — sürekli entegrasyon)
Sentry	Canlıdaki hataları yakalayıp sana bildiren servis

Deploy (yayına alma): kodu, kullanıcıların erişebileceği bir sunucuya yükleyip çalıştırmak.

Bölüm 4 — /yeni-proje sekiz adımda ne yapar

Adım 0 — Bağımlılıklar

Kit tek başına çalışmaz; iki dış parçayı çağırır.

Addy Osmani'nin agent-skills paketi — İngilizce, 25 skill'lik bir kütüphane. Kit sekizini kullanır:

Skill	Ne yaptırır
interview-me	Tek tek soru sorarak asıl isteneni çıkarır
frontend-ui-engineering	Arayüzün "yapay zekâ işi" görünmesini engeller
test-driven-development	Önce başarısız test, sonra kod
security-and-hardening	Güvenlik açığı denetimi
code-review-and-quality	Kodu beş açıdan inceler
source-driven-development	Ezberden değil, resmi dokümandan yazar
browser-testing-with-devtools	Gerçek tarayıcıda kontrol
debugging-and-error-recovery	Hatayı sistematik bulma
incremental-implementation	Küçük dilimler hâlinde ilerleme
doubt-driven-development	Riskli kararı ikinci kez sorgulama

Bir çakışma olursa **kitin kendi kuralı üstündür** — çünkü kitinki Türkçe ve bu projeye özel.

chrome-devtools MCP — Claude'un tarayıcıyı fiilen kullanmasını sağlar: sayfayı açar, tıklar, ekran görüntüsü alır, hataları okur. **MCP** (Model Context Protocol): yapay zekâya dış araç bağlama standardı.

Adım 1 — Proje tipi

Sorular: **Bu proje kimin için?** (kendi projen mi, kurum projesi mi — sonraki her şeyi bu belirler) · web mi mobil mi · sunucu tarafı nasıl kurulacak.

Adım 2 — Kural dosyalarını yerleştir

19 standart, ajanın uyacağı kurallar dosyası (`CLAUDE.md`) ve senin okuyacağın kılavuz projeye kopyalanır.

Adım 3 — PRD

PRD (Product Requirements Document — ürün gereksinim belgesi): ne yapılacağını, kimin için yapılacağını ve **ne yapılmayacağını** yazan belge.

Burada tek tek soru sorulur. Netleşmeden geçilmez: kim kullanacak · hangi problemi çözüyor · **kapsam dışı ne** · roller · iş kuralları · aynı anda kaç kişi girecek · arka planda çalışacak iş var mı.

Değer sorusu: her özellik için "yapılmalı mı" sorulur — "Bu ekran hangi problemi çözüyor? Olmasaydı kullanıcı ne yapardı?" Kurum projelerinde analiz birimi çoğu zaman **çözümü** yazar, **problemi** değil.

Görüşmenin sonunda tasarım yönü ve arama motoru kapsamı sorulur.

Adım 4 — Yol haritası

Roadmap (yol haritası): hangi işin hangi sırayla yapılacağı. Sıra keyfi değil, **bağımlılığa** göredir: veritabanı olmadan API yazılmaz.

ADR (Architecture Decision Record — mimari karar kaydı): "neden böyle yaptık" sorusunun altı ay sonraki cevabı. Yazılmazsa sonraki oturum kararı "yanlışlıkla böyle olmuş" sanıp geri alır.

Yol haritası yazıldıktan sonra **altı gözle denetlenir:** ürün · risk · geri alınabilirlik · dış bağımlılık · mühendislik · kullanım.

Adım 5 — İskeleti kur

Çatı kurulur, tip denetimi açılır, biçimlendirme araçları ayarlanır, **git** başlatılır.

Git: kodun her hâlini saklayan sistem. Bozarsan geri dönersin. **Commit:** bir değişikliği "bu hâlini kaydet" demek. **Repo (depo):** projenin tüm geçmişiyle birlikte durduğu yer.

Sonra: **çalışan şey gösterilir.** Sayfa açılır, ekran görüntüsü alınır, veritabanı tabloları gösterilir. Bunu görmeden hiçbir hesap açılmaz.

Adım 6 — Yayın

Kimin için yaptığına göre ikiye ayrılır:

Kendi projen: GitHub deposu, yayın servisi, veritabanı, otomatik testler, sağlık kontrolü ucu, **bölge eşleşmesi** ve arama motoruna tanıtmak.

Bölge eşleşmesi: sunucu ile veritabanı aynı kıtada olmalı. Sunucu Amerika'da veritabanı Almanya'daysa her sorgu okyanus geçer, sayfa 1–2 saniye geç açılır.

Kurum projesi: yayına alınmaz. DevOps ekibinin çalıştıracağı **teslim paketi** hazırlanır ve kendi bilgisayarında denir. **DevOps:** sunucuları ve yayın süreçlerini yöneten ekip.

Adım 7 — Son kontrol

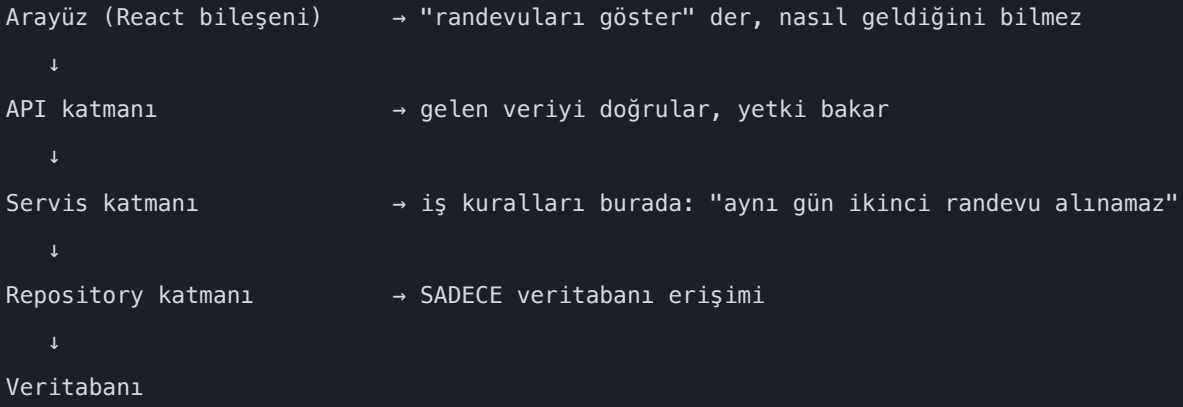
On yedi maddelik liste. Şablonlar dolduruldu mu, tasarım kararı yazıldı mı, ekran görüntülerine **bakıldı** mı, arama motoru ayarları kuruldu mu, bölgeler eşleşti mi.

Bölüm 5 — On dokuz kural dosyası

#	Konu	Ne diyor
00	Stack	Hangi teknoloji, neden. Başlangıç noktası — her projede yeniden ölçülür
01	Mimari	Kodun katmanları ve sırası
02	Kod standartları	Adlandırma, yorum, hata yönetimi
03	API	Sunucu ile arayüzün anlaşma biçimi
04	Veritabanı	Tablo, ilişki, hızlandırma, şema değişikliği
05	Güvenlik	Giriş, oturum, yetki
06	Test	Neyi nasıl test ederiz, beş gözle doğrulama
07	Arayüz	Renk, boşluk, karanlık tema, hareket, erişilebilirlik
08	Git	Kayıt biçimi, dal, geri alma
09	CI/CD	Otomatik test ve yayın
10	Definition of Done	"Bitti" ne demek
11	Ajanla çalışma	Yapay zekânın uyacağı davranış kuralları
12	İşletme	İzleme, log, ani yük
13	Ortamlar	Kendi bilgisayarın · deneme · canlı
14	Gizlilik/KVKK	Kişisel veri, hesap silme
15	Oturum devri	Sonraki oturuma bilgi aktarma
16	Kurulum	Kurulum protokolü
17	Mobil	Telefon uygulaması
18	SEO	Arama motorunda bulunma

01 — Mimari: katman sırası

Bu, kitin en önemli kurallarından biri.



Katman: işin belirli bir parçasından sorumlu kod bölümü. **Repository (depo katmanı):** veritabanı sorgularının yazıldığı **tek** yer. Bir araç değil, senin yazdığın kod katmanıdır; aracı Prisma'dır.

Kural: katman atlanmaz. Ekran içinden doğrudan veritabanına gidilmez.

Neden: iş kuralını değiştirmek gerektiğinde tek yere bakarsın; veritabanını değiştirmek gerektiğinde sadece alt katmanı yazarsın; iş kuralını test ederken veritabanına hiç dokunmazsın.

03 — API ve sözleşme

API: iki yazılımın konuşma yolu. Arayüz "*bana randevuları ver*" der, sonucu verir.

Endpoint (uç): o konuşmanın belirli bir adresi — `/api/randevular` gibi.

Contract (sözleşme): "*bu uç ne alır, ne döner*" anlaşması. Kitle `packages/contracts` klasöründe Zod şeması olarak tek yerde durur; iki taraf da aynısını kullanır.

Faydası: sunucuda bir alan adını değiştirirsen arayüz **derlenmez**. Hata sen fark etmeden canlıya gidemez. Sözleşme olmasa hata kullanıcının ekranında çıkardı.

06 — Beş gözle doğrulama

Testlerin yeşil olması "bitti" demek değildir. Otomatik test **kodun yaptığını** doğrular, **doğru şeyi yaptığını** değil. Bir özellik bitince sırayla:

Göz	Bakılan	Kim bakar
Backend	Mutlu yol, hata yolları, yetkisiz erişim, sınır değerler	Ajan kanıt sunar
Veri	Kayıt gerçekten yazıldı mı — tablolar açılıp bakılır	Sen görürsün
Frontend	Üç ekran genişliği, iki tema, konsol hatası	Ekran görüntüleri
Tasarım/UX	"Çalışıyor ama kullanıcı açısından saçma"	Asıl senin katmanın
Güvenlik	Yetki, girdi doğrulama, hata mesajı sızıntısı	Ajan kanıt sunar

Backend: sunucu tarafı. **Frontend:** kullanıcının gördüğü taraf. **UX** (User Experience — kullanıcı deneyimi): kullanmanın nasıl hissettirdiği. **Konsol:** tarayıcının hata gösterdiği gizli pencere.

Ayrıca **etki alanı** yazılır: bu değişiklik başka hangi ekranları, hangi API uçlarını, hangi eski kayıtları etkiledi. "*Sadece şu dosyaya dokundum*" cevap sayılmaz.

Ve **öğretme zorunluluğu**: ajan ne kontrol ettiğini **ve neden o kontrolü yaptığını** anlatır. "*Test geçti*" tek başına rapor değildir — neyin test edildiği söylenmezse neyin test **edilmediği** bilinemez.

07 — Arayüz: "yapay zekâ işi" görünmemesi

Yapay zekâ, karar verilmemiş her yerde kendi varsayılanına düşer ve bütün uygulamalar birbirine benzer. Kit bunu iki şekilde engeller:

Önce karar: kod yazılmadan yazı tipi, renk paleti ve ürünün karakteri kararlaştırılır, ADR'ye yazılır.

Sonra yasak liste: mor gradyan, her şeye maksimum yuvarlak köşe, gölge bombardımanı, tek tip kart ızgarası, şişkin boşluk, uydurma metin.

Gradyan: iki rengin birbirine geçtiği geçiş efekti.

Ayrıca zorunlu olanlar: **responsive** (ekran boyutuna göre uyum), **karanlık tema**, **erişilebilirlik** — klavyeyle kullanılabilme, yeterli renk kontrastı, ekran okuyucu desteği. Bunlar görme engelli veya fare kullanamayan kullanıcılar için, ve kamu hizmetlerinde yasal beklenti.

12 — Ani yük

Belediye başvurusu saat 12:00'de açılır ve 20.000 kişi aynı anda girer. Buna **spike traffic** (ani patlama trafiği) denir ve plansız girilirse sistem o dakikada çöker.

Yaygın yanılgı, sunucu yazılımını hızlandırmaktır. Gerçek darboğaz **veritabanı bağlantı sayısıdır**. Beş zorunlu önlem: bağlantı havuzu · yazma işlerini kuyruğa alma · okuma sayfalarını önceden hazırlama · hız sınırı · kapasite aşılsa bekleme ekranı.

Bağlantı havuzu (connection pool): binlerce isteği az sayıda gerçek veritabanı bağlantısına indiren ara katman. **Kuyruk (queue)**: hemen yapılması gerekmeyen işlerin sıraya konduğu yapı — e-posta gönderme, PDF üretme gibi. Kullanıcı beklemez.

18 — SEO

SEO (Search Engine Optimization): sitenin arama motorunda bulunabilmesi.

En kritik karar **render stratejisi**: sayfanın içeriği sunucuda mı hazırlanıyor, tarayıcıda mı? Arama motoru robotu tarayıcıda üretilen içeriği çoğu zaman göremez. İçerik sunucuda hazırlanmalı.

Render: sayfanın görünür hâle gelmesi.

Diğerleri: her sayfanın kendine özgü başlığı ve açıklaması · adres biçimi (Türkçe karakter kullanılmaz) · **sitemap.xml** (sayfa listesi) · **JSON-LD** (arama motoruna "*bu bir ürün, fiyatı şu*" diyen görünmez etiket) · yayından sonra Google'a haber verme.

Bölüm 6 — Yapay zekânın uyduğu kapılar

CLAUDE.md, ajanın **neyi yapamayacağını** söyleyen dosya. En önemli üçü:

Kapı	Kural	Neden
1	Kodu okuyup " <i>çalışması lazım</i> " demek yetmez — tarayıcıda açıp göster	Kod doğru görünüp çalışmayabilir
2	Plan sun, onay bekle — kod yazmadan önce, her zaman	Yanlış varsayımla yazılmış 200 satır, sorulmuş bir sorudan pahalı
3	Bir ADR'ye aykırı kod yazılmaz; karar değişecekse önce yeni ADR	Yoksa kararlar sessizce erir

Ve iş bölümü: **ajanın yapabildiği hiçbir iş kullanıcıya yaptırılmaz**. Senin zamanın yalnızca ajanın *yapamadığı* işler için harcanır — hesap açma, ödeme, kurumdan yetki alma. Bunlar kimlik doğrulaması gerektirir.

Bir kural daha: **üçüncü başarısız düzeltmeden sonra kod yazılmaz**. Aynı hata için üç yama tutmadıysa sorun yanlış tahmin değil, **yanlış yapıdır**. Ajan durur ve mimariyi tartışmaya açar.

Bölüm 7 — Kit nasıl büyüyor

Kit gerçek projelerdeki hatalardan büyür:

Projede bir hata yapılır veya daha iyi bir yol bulunur

↓

/kit-senkron çalıştırılır

↓

Fark üç kutudan birine konur:

- Her projede geçerli → kite yazılır
- Yalnızca bu projeye → projede kalır
- Kit ilerlemiş → projeye getirilir

↓

Sürüm numarası artırılır, GitHub'a gönderilir

Kural: bir kural projeye özel hâle geliyorsa o kural **yanlış yazılmıştır**. Kural düzeltilir, projeye göre dallandırılmaz. Dallandırma bir framework'ü çürütür: üç proje sonra birbirinden sapmış üç kopya olur ve hangisinin doğru olduğu bilinmez.

Sözlük

Terim	Türkçesi / Anlamı
API	İki yazılımın konuşma yolu
ADR	Mimari karar kaydı — "neden böyle yaptık"
Backend	Sunucu tarafı, kullanıcının görmediği kısım
CI/CD	Her değişiklikte otomatik test ve yayın
Commit	Bir değişikliği kaydetme
Container / Konteyner	Uygulamayı gereken her şeyle birlikte saran kutu
Contract / Sözleşme	API'nin ne alıp ne döneceği anlaşması
Deploy	Yayına alma
DevOps	Sunucu ve yayın süreçlerini yöneten ekip
Endpoint / Uç	API'nin belirli bir adresi
Framework / Çatı	Hazır iskelet
Frontend	Kullanıcının gördüğü taraf
Git / Repo	Kodun tüm geçmişini saklayan sistem
Index	Veritabanında arama hızlandırıcı
JSON-LD	Arama motoruna sayfayı tarifleyen görünmez etiket
Katman	İşin bir parçasından sorumlu kod bölümü
Kuyruk / Queue	Sonra yapılacak işlerin sırası
MCP	Yapay zekâya dış araç bağlama standardı
Migration	Veritabanı yapısının adım adım değiştirilmesi
ORM	Kod ile veritabanı arasındaki çevirmen
PRD	Ürün gereksinim belgesi
Render	Sayfanın görünür hâle gelmesi
Repository	Veritabanı sorgularının yazıldığı tek katman
Responsive	Ekran boyutuna göre uyum sağlama
Roadmap	Yol haritası
Serverless	Sunucuyu sürekli açık tutmadan, istek geldikçe çalışan yapı
Servis katmanı	İş kurallarının yazıldığı yer
SEO	Arama motorunda bulunabilirlik
Sitemap	Sitedeki sayfaların listesi
Skill	Yapay zekâya iş tarifi veren talimat dosyası
Spike traffic	Ani trafik patlaması

Terim	Türkçesi / Anlamı
Terminal	Bilgisayara yazıyla komut verilen ekran
TypeScript	JavaScript'in tip denetimli hâli
UX	Kullanıcı deneyimi
Erişilebilirlik (a11y)	Engelli kullanıcıların da kullanabilmesi